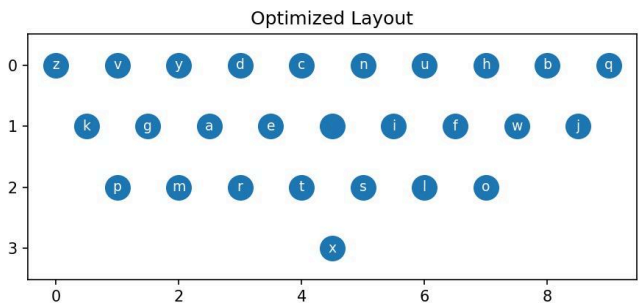
APL ASSIGNMENT 3 - Keyboard optimisation

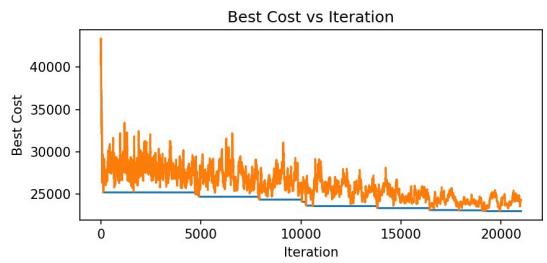
EE24B092
Avisha Mathur

Ran simulated annealing on some sample text (the text was a bunch of python programs concatenated).
Here are the plots:

1.
```
Baseline (QWERTY assignment) cost: 42815.7132
params : iters = 21000, intial temp = 600.0, cooling rate = 0.993, epoch = 100
```

```
Optimized cost: 22937.8334  (improvement 19877.8798)
Runtime: 270.59s over 21000 iterations
```

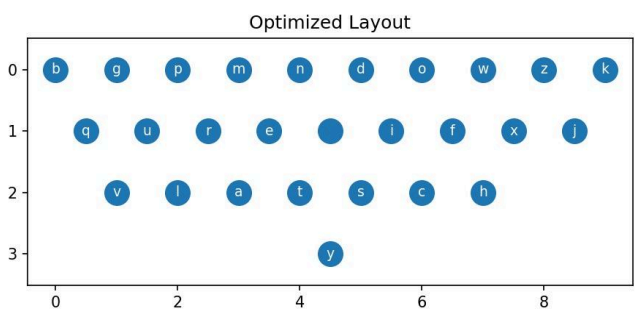


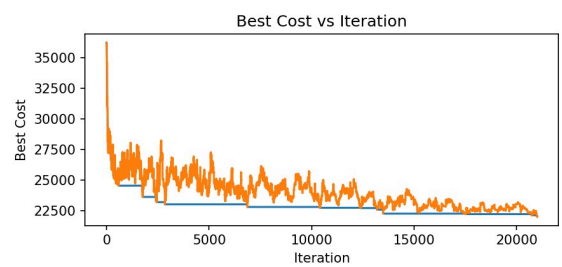


2. with lesser temp, same amount of iterations

```
PS C:\Users\avish> python "C:\Users\avish\Downloads\kbd_optim_base.py"
Baseline (QWERTY assignment) cost: 36218.5271
params : iters = 21000, intial temp = 300.0, cooling rate = 0.993, epoch = 100
```

```
Optimized cost: 21995.5457  (improvement 14222.9814)
Runtime: 169.64s over 21000 iterations
```



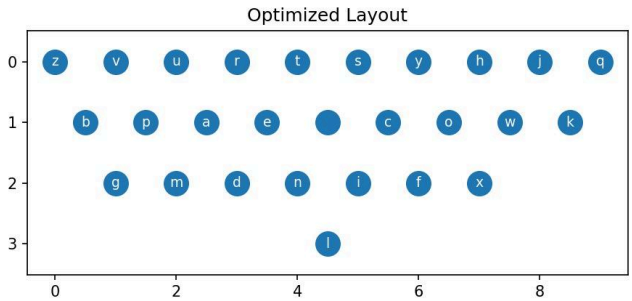


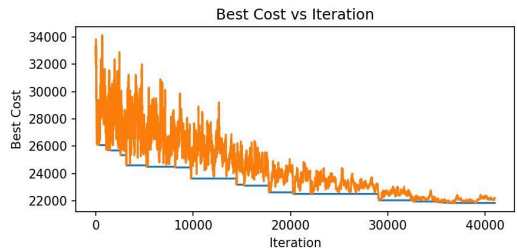
As we can observe, lesser temperature gives lesser freedom to try out layouts far from the best layout we have found so far. So you can see less variation of the orange graph (current cost) and blue graph (best cost so far).

3. same temp, diff number of iterations

```
Baseline (QWERTY assignment) cost: 33243.7879
params : iters = 41000, intial temp = 600.0, cooling rate = 0.993, epoch = 100
```

```
Optimized cost: 21855.6560  (improvement 11388.1320)
Runtime: 483.66s over 41000 iterations
```





We ended up with (marginally) lesser cost. This is because if it runs for more iterations, it naturally has more chances of converging to a more optimal solution, since we are trying randomly generated layouts.

Steps to run the code (command line)

python kbd_optim.py [SAMPLE TEXT FILE PATH] –iters [NUMBER OF ITERATIONS] –temp [INITIAL TEMPERATURE] –alpha [COOLING RATE] –epoch [NUMBER OF ITERATIONS PER TEMPERATURE STEP]

Like this -

```
C:\Users\avish>python C:\Users\avish\Downloads\kbd_optim_base.py C:\Users\avish\TrainingText.txt --iters 30000 --temp 400 --alpha 0.9
95 --epoch 150
Baseline (QWERTY assignment) cost: 36758.1141
params : iters = 30000, intial temp = 400.0, cooling rate = 0.995, epoch = 150
```